

Vue.js (Data)

Interface Pessoa-Máquina - 25/26 - LEI / UM

Hugo Pacheco
hpacheco@di.uminho.pt

Condicionais

- Nos templates HTML do Vue, podemos renderizar elementos condicionalmente

```
<h1 v-if="awesome">Vue is awesome!</h1>
```

- Suporte para condições encadeadas

```
<div v-if="type == 'A'"> A </div>  
<div v-else-if="type == 'B'"> B </div>  
<div v-else-if="type == 'C'"> C </div>  
<div v-else> Not A/B/C </div>
```

! Têm que ser consecutivas

- ? Qual a diferença entre `v-if` e `v-show`?

```
<h1 v-show="awesome">Vue is awesome!</h1>
```

💡 Apenas faz *toggle* da propriedade `display` no CSS

Iteração

- Podemos iterar sobre uma lista e construir uma sequência de elementos

```
const items =  
[ { message: 'Foo' },  
  { message: 'Bar' } ]
```

```
<li v-for="item in items">  
  {{ item.message }}  
</li>
```

```
<li v-for="(item,index) in  
items">  
  {{ index }}:{{ item }}  
</li>
```

- Também podemos iterar sobre um objeto

```
const myObject = {  
  title: 'Test',  
  author: 'Vue'  
})
```

```
<li v-for="value in myObject">  
  {{ value }}  
</li>
```

```
<li v-for="(value,key) in  
myObject">  
  {{ key }}:{{ value }}  
</li>
```

Exemplo (InsideAirBnb Listings)

- Visualizar tabela de dados (simulados) do Inside AirBnb
 - Two-way data binding
 - Vários input types
- Editar células
- Inserir / remover linhas



💡 Mudámos de setup

💡 Projeto com vite

Estrutura

- main.js
- App.vue

InsideAirBnb Listings												
Action	id	name	host_id	host_name	neighbourhood_group	neighbourhood	latitude	longitude	room_type	price	minimum_nights	number_of_reviews
-	41339	Porto city fl	180050	Paula	PORTO	Lordelo do Ouro	41,1501	-8,66035	Entire home	69	2	33
-	55111	Fontielas H	259711	Isabel E Jo�	PAREDES	Cete	41,1741�	-8,35533	Entire home		5	21
-	73828	Fontielas H	259711	Isabel E Jo�	PAREDES	Cete	41,1762�	-8,35351	Entire home		5	20
+												

Iteração (keys)

- É boa prática definir um atributo `:key` para cada `v-for` (documentação)
 - 💡 Define um identificador único que controla quais os dados Vue associados a cada elemento HTML
 - 💡 Relevante para a renderização do Virtual DOM
- Sem `:key` = updates “in-place”
 - “This default mode is efficient, but **only suitable when your list render output does not rely on child component state or temporary DOM state (e.g. form input values)**” (Vue documentation)
- Com `:key` = estado interno associado a cada key
 - ! Inconsistências se keys não únicas
- 💡 Valores das keys devem ter tipos JS primitivos

Exemplo (InsideAirBnb Listings + Keys)

- Podemos acrescentar um campo de notas a cada linha + shuffling de linhas

💡 Diferentes comportamentos consoante a `:key`, e.g.:

```
<tr v-for="(row, row_index) in listings">
```

```
<tr v-for="(row, row_index) in listings" :key="row_index">
```

```
<tr v-for="(row, row_index) in listings" :key="row.id">
```

InsideAirBnb Listings				
Action	id	name	host_id	host_name
- Add notes	41339	Porto city fl	180050	Paula
- Add notes	55111	Fontielas H	259711	Isabel E Joã
- Add notes	73828	Fontielas H	259711	Isabel E Joã
+				
Shuffle Listings				

JSON server

- Uma ferramenta que cria uma API de *mockup* a partir de um ficheiro JSON
 - Sem dependências, para rápido deployment
 - Minimalista, para prototipagem de front-ends
- Suporta operações CRUD (Create, Read, Update, Delete)
- Utilização:
 1. Criar um ficheiro `db.json` com os dados
 2. `json-server db.json`
 3. REST API à escuta em `localhost:3000`

JSON server (db . j s o n)

💡 Para a 2ª fase do trabalho prático

- Podem criar um único ficheiro db . j s o n a partir dos dados recolhidos
 - Podem utilizar este script Python que cria um base de dados JSON a partir de uma hierarquia de pastas com ficheiros CSV e JSON
 - Podem acrescentar ao ficheiro JSON resultante toda a informação adicional que seja útil para o vosso protótipo
- ! JSON é um formato bastante pouco compacto. Filtrem primeiro a informação relevante para evitar uma base de dados de muitos MB
- ! O propósito dos dados é **sempre e apenas para *mockup***
 - ! Dados devem ser representativos, mas não necessariamente fidedignos
 - ! Não se preocupem com ter listagens demasiado completas

JSON server (REST)

- REST (REpresentational State Transfer)
 - Acesso a serviços web de forma simples e flexível
 - Stateless: Cada operação é independente
- Protocol HTTP para efetuar pedidos ao servidor



- Operações CRUD através de métodos HTTP

POST localhost:3000/listings
{...}

GET localhost:3000/listings/

GET localhost:3000/listings?
id=73828

PUT localhost:3000/listings
{...}

PATCH localhost:3000/listings
{...}

DELETE localhost:3000/
listings/aa6e

Exemplo (InsideAirBnb Listings + DB)

- Ações sincronizadas com DB json-server
 - Inserir / remover linha
 - Atualizar coluna
- 💡 Assíncrono: estado do front-end independente de estado da DB

InsideAirBnb Listings							
Action	id	name	host_name	neighbourhood_group	price	last_review	license
-	41339	Porto city flat	Paula	PORTO	69	09/09/2025 📅	29049/AL
-	55111	Fontielas Hou	Isabel E João	PAREDES		30/08/2025 📅	7563/AL
-	73828	Fontielas Hou	Isabel E João	PAREDES		31/08/2025 📅	7563/AL
-	87873	Oporto Apartn	Paula	PORTO	108	14/09/2025 📅	634/AL
-	94701	Big TERRACE	Gundega	PORTO	73	15/09/2025 📅	85541/AL

Fetch API (JS)

- Interface para fazer pedidos HTTP
- Sintaxe: `fetch(url, options)`
 - `url`: the URL of the resource being requested
 - `options`: an optional options object to configure the request

```
{ method: 'POST',  
  headers: { 'Content-Type':  
    'application/json' },  
  body:  
    JSON.stringify({ key1:  
      'value1', key2: 'value2' }),  
  ...  
}
```

- Métodos:
 - GET: Retrieve data from a server (default method)
 - POST: Send data to a server to create a new resource
 - PUT: Send data to a server to update an existing resource
 - DELETE: Remove an existing resource from the server
 - PATCH: Send partial data to update parts of an existing resource

Promises (JS)

- A função `fetch` retorna uma *promise* (um objeto que encapsula uma computação a ser executada assincronamente)
- Em JS, podemos encadear promises
 - `then()` deals with fulfilled promises
 - `catch()` deals with rejected promises
 - `finally()` executes after the promise is resolved (regardless of its outcome)
- `await` espera pelo resultado de uma promise
- Podemos declarar uma `async function`, que retorna uma promise
 - 💡 Intuição: o browser não fica à espera do resultado